

Laporan Jurnal Praktikum

Pengembangan Aplikasi

Berbasis Web

**Modul 4: Mengoperasikan Database dan
Generate Data Otomatis dengan Laravel**



Nama : Ryandi Agung Mazhendra

Nim : 707012500024

Kelas: 49-04

D4SIKC

4.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep koneksi database pada Laravel.
2. Membuat koneksi database melalui file .env.
3. Membuat dan menjalankan migration untuk membentuk tabel database.
4. Membuat form input data mahasiswa.
5. Melihat dan memastikan data tersimpan ke dalam database.
6. Mampu mengimplementasikan database seeding untuk menghasilkan data secara otomatis dalam database.

4.2 Alat dan bahan

Untuk menjalankan modul ini, diperlukan:

Perangkat Keras :

- Komputer atau laptop dengan sistem operasi Windows/Linux/macOS.
- Minimal prosesor 1 GHz, RAM 4GB (disarankan 8GB), serta ruang penyimpanan kosong minimal 2GB.

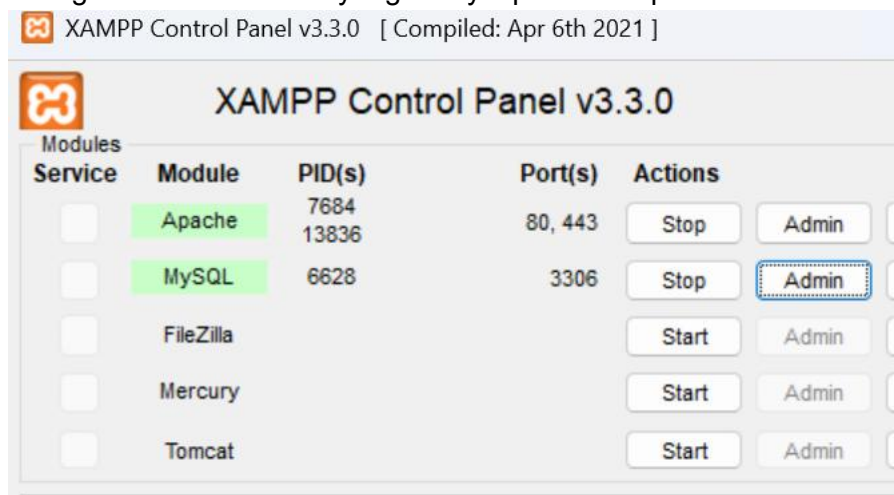
Perangkat Lunak:

- XAMPP atau Laragon sebagai server lokal (Apache & MySQL).
- Composer sebagai dependency manager PHP.
- PHP versi 8.0 atau lebih tinggi (sudah termasuk dalam XAMPP/Laragon).
- Laravel Framework (minimal versi 8) yang sudah terinstal melalui Composer.
- Teks Editor / IDE seperti Visual Studio Code, Sublime Text, atau PHPStorm.
- Browser Web seperti Google Chrome, Safari, atau Mozilla Firefox untuk menjalankan aplikasi.

4.3 Langkah-langkah Praktikum

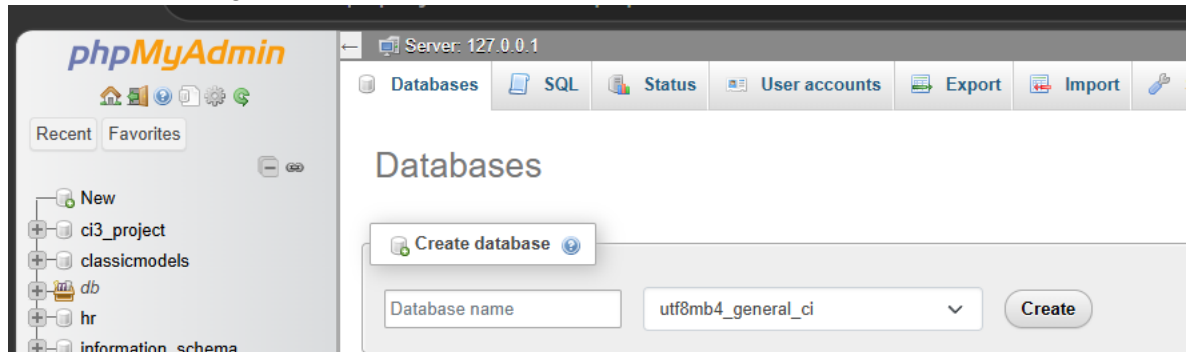
1. Menyalakan Server Lokal

Buka XAMPP Control Panel, lalu klik Start pada modul Apache dan MySQL sampai keduanya berwarna hijau. Apache berfungsi sebagai web server, sedangkan MySQL sebagai server database yang menyimpan data aplikasi.



2. Membuat Database

Buka <http://localhost/phpmyadmin>, database ini masih kosong karena tabelnya akan dibuat melalui migration.



3. Konfigurasi Koneksi Database(.env)

Buka file .env di folder utama project, lalu ubah bagian database seperti berikut. Pastikan tanda # di depan baris DB_HOST sampai DB_PASSWORD dihapus.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laravel
DB_USERNAME=root
DB_PASSWORD=
```

4. Membuat Migration

Jalankan perintah berikut di terminal:

```
php artisan make:migration create_mahasiswa_table
```

Perintah ini membuat file di folder database/migrations dengan awalan tanggal dan waktu yang dipakai Laravel untuk menentukan urutan eksekusi. Kemudian mengisi file tersebut dengan kode berikut:



Method up() dijalankan saat migrate untuk membuat tabel, sedangkan down() dijalankan saat rollback untuk menghapus tabel.

5. Membuat Model

Jalankan kode ini di terminal :

php artisan make: model mahasiswa

Kemudian ganti isi file kode nya seperti gambar berikut



```
1 class Mahasiswa extends Model
2 {
3     protected $table = 'mahasiswa';
4     protected $fillable = [
5         'nama',
6         'email',
7         'umur',
8         'jurusan',
9     ];
10 }
```

Properti \$table ditulis eksplisit karena Laravel secara bawaan menganggap nama tabel adalah bentuk jamak dari nama model (mahasiswas). Properti \$fillable berisi kolom yang boleh diisi lewat mass assignment.

6. Membuat Controller

Jalankan kode ini di terminal :

php artisan make : controller mahasiswaController

Kemudian isi filenya diganti dengan kode berikut:



```
1 class MahasiswaController extends Controller
2 {
3     public function form()
4     {
5         return view('form');
6     }
7     public function simpan(Request $request)
8     {
9         Mahasiswa::create([
10             'nama' => $request->nama,
11             'email' => $request->email,
12             'umur' => $request->umur,
13             'jurusan' => $request->jurusan,
14         ]);
15         return "Data mahasiswa berhasil disimpan.";
16     }
17     public function daftar()
18     {
19         $data = Mahasiswa::all();
20         return view('mahasiswa', ['mahasiswa' => $data]);
21     }
22 }
```

7. Membuat Route

Membuka file web.php di router kemudian menambahkan kode berikut

```
1 Route::get('/form', [MahasiswaController::class, 'form']);
2 Route::post('/simpan', [MahasiswaController::class, 'simpan']);
3 Route::get('/daftar-mahasiswa', [MahasiswaController::class, 'daftar']);
```

Route /simpan memakai Route::post karena menerima data dari form, sedangkan dua route lainnya memakai Route::get karena hanya menampilkan halaman.

8. Membuat View

a) File resources/views/form.blade.php

```
1 <body>
2   <h1>Input Mahasiswa</h1>
3
4   <form action="/simpan" method="post">
5     @csrf
6     <label for="nama">Nama:</label>
7     <input type="text" name="nama" id="nama" required><br><br>
8
9     <label for="email">Email:</label>
10    <input type="email" name="email" id="email" required><br><br>
11
12    <label for="umur">Umur:</label>
13    <input type="number" name="umur" id="umur" required><br><br>
14
15    <label for="jurusan">Jurusan:</label>
16    <input type="text" name="jurusan" id="jurusan" required><br><br>
17
18    <button type="submit">Simpan</button>
19  </form>
20 </body>
```

b) File resources/views/mahasiswa.blade.php

```

1 <body>
2 <h2>Daftar Mahasiswa</h2>
3 <table border="1" cellpadding="5" cellspacing="0">
4 <thead>
5 <tr>
6 <th>ID</th>
7 <th>Nama</th>
8 <th>Email</th>
9 <th>Umur</th>
10 <th>Jurusan</th>
11 </tr>
12 </thead>
13 <tbody>
14 @foreach($mahasiswa as $mhs)
15 <tr>
16 <td>{{ $mhs->id }}</td>
17 <td>{{ $mhs->nama }}</td>
18 <td>{{ $mhs->email }}</td>
19 <td>{{ $mhs->umur }}</td>
20 <td>{{ $mhs->jurusan }}</td>
21 </tr>
22 @endforeach
23 </tbody>
24 </table>
25 </body>
26 </html>

```

9. Membuat Seeder

Jalankan kode ini di terminal :

php artisan make:seeder StudentSeeder

File StudentSeeder.php akan muncul di folder database/seeder. Di folder yang sama terdapat DatabaseSeeder.php, yaitu seeder utama yang dapat memanggil seeder lain melalui `$this->call([StudentSeeder::class]);`.

10. Mengisi Seeder dengan Faker

Ganti isi file database/seeder/StudentSeeder.php dengan kode berikut:

```

1 class StudentSeeder extends Seeder
2 {
3     public function run(): void
4     {
5         $faker = Faker::create('id_ID');
6         for ($i = 0; $i < 20; $i++) {
7             DB::table('mahasiswa')->insert([
8                 'nama' => $faker->name,
9                 'email' => $faker->unique()->safeEmail(),
10                'jurusan' => $faker->randomElement(['Informatika', 'Sistem Informasi', 'Teknik Komputer']),
11                'umur' => $faker->numberBetween(18, 25),
12                'created_at' => now(),
13                'updated_at' => now(),
14            ]);
15        }
16    }
17 }

```

Karena kolom nim diganti email, isi \$fillable pada model, input pada form.blade.php, dan kolom pada mahasiswa.blade.php juga disesuaikan agar form dan halaman daftar tetap berfungsi.

11. Migrate Ulang dan Menjalankan Seeder

Menjalankan kode ini di terminal :

```
php artisan migrate:refresh
```

```
php artisan db:seed --class=StudentSeeder
```

Perintah migrate:refresh me-rollback seluruh tabel lalu menjalankan ulang semua migration, sehingga data lama ikut terhapus dan hanya aman dipakai pada tahap pengembangan. Kedua perintah tersebut dapat digabung menjadi satu perintah:

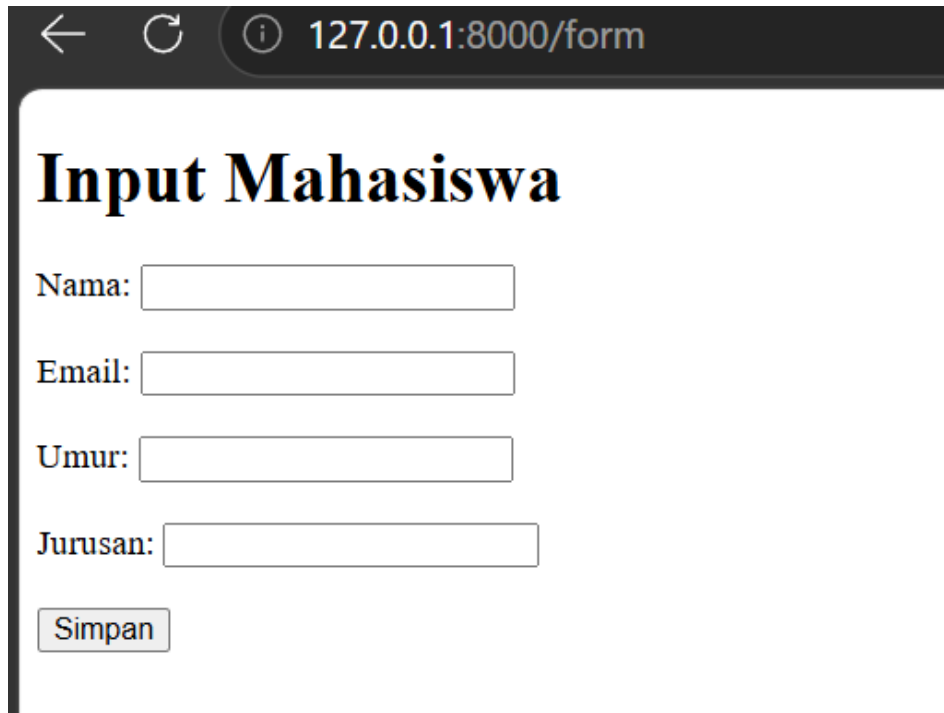
```
php artisan migrate:fresh --seed
```

Terakhir, buka tabel mahasiswa di phpMyAdmin untuk memastikan 20 data dummy sudah masuk.

☐	 Edit	 Copy	 Delete	1	Mulyono Widodo S.E.I	zizi77@example.net	Informatika	21	2026-10-05 02:49:49	2026-10-05 02:49:49
☐	 Edit	 Copy	 Delete	2	Ganda Pratama	msinaga@example.net	Sistem Informasi	21	2026-10-05 02:49:49	2026-10-05 02:49:49
☐	 Edit	 Copy	 Delete	3	Tina Nurul Pertiwi S.H.	gunawan.yunita@example.net	Informatika	21	2026-10-05 02:49:49	2026-10-05 02:49:49
☐	 Edit	 Copy	 Delete	4	Bakiono Tomi Januar	roktaviani@example.net	Sistem Informasi	22	2026-10-05 02:49:49	2026-10-05 02:49:49
☐	 Edit	 Copy	 Delete	5	Edward Jindra Saputra	szulkarnain@example.com	Teknik Komputer	25	2026-10-05 02:49:49	2026-10-05 02:49:49
☐	 Edit	 Copy	 Delete	6	Tina Vanesa Puspita	purwanti.raisa@example.org	Sistem Informasi	24	2026-10-05 02:49:49	2026-10-05 02:49:49
☐	 Edit	 Copy	 Delete	7	Ianaraga Mustafa	usarda.lana@example.net	Teknik Komputer	23	2026-10-05 02:49:49	2026-10-05 02:49:49

4.4 Hasil dan Dokumentasi Program

1. Hasil web form



← ↻ ⓘ 127.0.0.1:8000/form

Input Mahasiswa

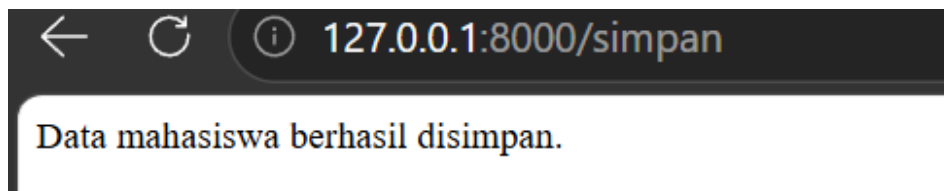
Nama:

Email:

Umur:

Jurusan:

2. Hasil konfirm data



← ↻ ⓘ 127.0.0.1:8000/simpan

Data mahasiswa berhasil disimpan.

3. Hasil we DaftarMahasiswa

127.0.0.1:8000/daftar-mahasiswa				
Daftar Mahasiswa				
ID	Nama	Email	Umur	Jurusan
1	Mulyono Widodo S.E.I	zizi77@example.net	21	Informatika
2	Ganda Pratama	msinaga@example.net	21	Sistem Informasi
3	Tina Nurul Pertiwi S.H.	gunawan.yunita@example.net	21	Informatika
4	Bakiono Tomi Januar	roktaviani@example.net	22	Sistem Informasi
5	Edward Jindra Saputra	szulkarnain@example.com	25	Teknik Komputer
6	Tina Vanesa Puspita	purwanti.raisa@example.org	24	Sistem Informasi
7	Jagaraga Mustofa	usada.lega@example.net	23	Teknik Komputer
8	Tari Kuswandari	twaskita@example.com	25	Informatika
9	Dimas Prasasta S.Pt	daliman79@example.com	21	Sistem Informasi
10	Iriana Zulfa Puspasari	ida73@example.net	25	Informatika
11	Talia Handayani	cemplunk.halim@example.org	18	Informatika
12	Gaduh Widodo	ina66@example.net	24	Sistem Informasi
13	Widya Cindy Yuliarti	yono.rahayu@example.org	22	Sistem Informasi
14	Vicky Eka Wulandari	mwastuti@example.com	23	Informatika
15	Ilsa Haryanti	bsamosir@example.net	25	Informatika
16	Shakila Handayani	kawaca04@example.net	23	Sistem Informasi
17	Rika Safitri S.Psi	lasmanto90@example.net	25	Teknik Komputer
18	Gabriella Zizi Laksita S.Kom	usyi60@example.com	23	Teknik Komputer
19	Wakiman Arsipatra Waskita S.Gz	kambali87@example.com	22	Informatika
20	Najwa Pertiwi	bpuspita@example.com	20	Teknik Komputer
21	ryandi Agung Mazhendra	ggwh@gmail.com	20	SIKC
23	udin	ytyti@gmail.com	20	SIKC

4.5 Analisis dan Pembahasan

Analisis. Aplikasi ini berjalan dengan alur form → route → controller → model → database. Pengguna mengisi form di /form, data dikirim dengan method POST ke route /simpan, lalu diteruskan ke method simpan() pada MahasiswaController yang menyimpannya ke tabel mahasiswa melalui model Mahasiswa. Saat /daftar-mahasiswa dibuka, method daftar() mengambil seluruh data dengan Mahasiswa::all() dan mengirimkannya ke view untuk ditampilkan. Berbeda dengan Modul 3 yang datanya berupa array di controller dan hilang saat halaman ditutup, kini data tersimpan permanen di database MySQL dan diakses sebagai objek (misalnya \$mhs->nama). Selama pengerjaan, kendala yang mungkin muncul antara lain koneksi database gagal karena MySQL belum aktif atau .env masih sqlite, galat 419 Page Expired karena @csrf belum ditulis, MassAssignmentException karena kolom belum masuk \$fillable, serta galat column not found saat seeding karena struktur tabel tidak

sesuai seeder, yang diselesaikan dengan menyesuaikan migration lalu menjalankan `migrate:refresh`.

Pembahasan. Migration berperan membuat dan mengelola struktur tabel melalui kode PHP sehingga lebih rapi, terdokumentasi, mudah dibagikan antaranggota tim, dan dapat dibatalkan dengan rollback, dibandingkan membuat tabel manual di phpMyAdmin. Eloquent ORM membuat penyimpanan dan pengambilan data cukup dengan `create()` dan `all()` tanpa menulis SQL, sedangkan `$table` dan `$fillable` memastikan model terhubung ke tabel yang benar dan membatasi kolom yang boleh diisi sebagai lapisan keamanan. Seeder bersama Faker mengisi tabel dengan data dummy secara otomatis, termasuk nama bernuansa Indonesia lewat locale `id_ID`. Dalam analogi rak buku, migration membangun rak (struktur tabel) dan seeder mengisinya dengan buku (data), sehingga pengujian tampilan dengan banyak data menjadi lebih cepat. Perlu diingat bahwa `migrate:refresh` menghapus seluruh data lama, jadi hanya cocok digunakan pada tahap pengembangan.

4.6 Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa Laravel dapat terhubung ke database MySQL dengan mengatur file `.env`, sehingga aplikasi dapat menyimpan dan membaca data secara permanen. Migration memudahkan pembuatan dan pengelolaan struktur tabel melalui kode tanpa membuat tabel manual di phpMyAdmin. Model dengan Eloquent ORM memungkinkan data disimpan dan diambil tanpa menulis perintah SQL secara langsung. Alur aplikasi `form` → `route` → `controller` → `model` → `database` berjalan dengan baik, dibuktikan dengan data dari form yang tersimpan di tabel dan tampil di halaman daftar. Seeder yang dikombinasikan dengan Faker berhasil membuat 20 data dummy secara otomatis sehingga proses pengujian aplikasi menjadi lebih cepat dan efisien.